

A Demo of the **session-paper** OpenCode Plugin: Automatic Session Distillation and Paper Generation

Author placeholder

September 24, 2026

Abstract

We present a basic demo and test of **session-paper**, an OpenCode server plugin registered at `~/.config/opencode/plugins/session-paper.ts`. The plugin injects a standing directive into every primary session’s system prompt instructing the active agent to distill the session and spawn the **latex-paper-writer** subagent at the end of every response, naming the output document from a slugified form of the session title, and it surfaces TUI toasts when the spawn is detected and when the child paper session finishes. A 14-check harness exercising the plugin’s three hooks passed in full, and the live end-to-end path was then exercised by spawning the subagent from a primary session, producing this document. We report the design under test, the harness checks, and the observed end-to-end results.

1 Introduction

Large language model agent sessions often produce useful structure—decisions, methods, and findings—that is lost when the conversation ends. Motivated by the desire to turn completed sessions into durable, citable documents, the **session-paper** plugin automates the last step of a session: distilling it into a LaTeX paper without requiring the user to prompt for this manually.

The plugin operates through three mechanisms working together: (i) a standing directive injected into the system prompt of primary sessions, (ii) detection of the moment the active agent spawns the **latex-paper-writer** subagent, and (iii) lifecycle tracking of the resulting child paper session, with user-visible TUI toasts marking the spawn and the child’s completion.

The research question for this demo is straightforward: *does the plugin function as designed across its three hooks, and does the automatic paper-generation loop complete end to end in a live session?* We answer it with a scripted test harness plus a live end-to-end run whose output is this very document.

2 Background and Related Work

The plugin builds on OpenCode’s extension points: registration in the OpenCode configuration, hooking of chat system-prompt transformation, interception of tool execution, and subscription to session events. Related in spirit are prior efforts to auto-generate scholarly artifacts from agent transcripts, but our scope here is narrower and strictly empirical: verify the plugin’s own contract, not the broader quality of model-generated papers. No external literature was surveyed for this demo; references are limited to the artifacts actually exercised.

3 Methods / Approach

3.1 Plugin under test

The plugin file `~/.config/opencode/plugins/session-paper.ts` was registered in the OpenCode configuration and confirmed active: its directive appeared in the session’s system prompt, with the slug `testing-session-paper-plugin-demo` derived from the session title “Testing session-paper plugin demo.”

The plugin’s three hooks are:

1. `experimental.chat.system.transform` — injects the standing directive into primary sessions only.
2. `tool.execute.after` — raises an info toast when a task tool spawns a subagent with `subagent_type` equal to `latex-paper-writer`.
3. An event handler — tracks `session.created/session.updated` titles, links the child paper session to its parent, and raises a success toast on `session.idle`.

3.2 Test harness

A 14-check `bun test` harness with a mock client exercised all three hooks. Beyond the happy path, the harness asserted negative and edge behaviors:

- the directive is suppressed in child/subagent sessions, preventing recursion;
- unrelated tools spawn no toast;
- unknown sessions fall back to `client.session.get`;
- renamed sessions update the derived slug;
- idle toasts fire exactly once per child session.

3.3 Live end-to-end run

Finally, the real path was exercised: the `latex-paper-writer` subagent was spawned from the primary session, the plugin detected the spawn and toasted, and the child session produced this document.

4 Results / Findings

All 14 harness checks passed, covering directive injection, slug derivation, child-session gating, and the toast lifecycle. Table 1 summarizes the hook-level outcomes.

The live end-to-end path also succeeded: the subagent spawn was detected and toasted, and this document was written to disk as `testing-session-paper-plugin-demo.pdf` with source `testing-session-paper-plugin-demo.tex`.

Hook	Behavior verified	Outcome
<code>system.transform</code>	Directive injected into primary sessions only	Pass
<code>tool.execute.after</code>	Toast on <code>latex-paper-writer</code> spawn only	Pass
Event handler	Title tracking, child linking, single idle toast	Pass

Table 1: Hook-level results from the 14-check harness.

5 Discussion

The results indicate the plugin works as designed. The negative checks matter as much as the positive ones: suppressing the directive in child sessions prevents a paper-writing subagent from recursively spawning further paper writers, and gating toasts on the specific `latex-paper-writer` subagent type avoids noise from unrelated tool calls. The rename check confirms slug derivation tracks live title edits rather than a stale snapshot, and the exactly-once idle toast prevents duplicated completion notifications.

This demo deliberately limits its claims. It verifies the plugin’s contract and one live run; it does not evaluate paper quality across sessions, robustness under concurrent sessions, or behavior when the subagent fails. Those remain open for future testing.

6 Conclusion

The `session-paper` plugin is operational: directive injection, slug derivation, child-session gating, and the toast lifecycle were all verified by a 14-check harness, and the automatic paper-generation loop functions end to end, as demonstrated by the production of this document.

Acknowledgments

References

- [1] `session-paper` OpenCode server plugin source, `~/config/opencode/plugins/session-paper.ts`; exercised in this session via its configuration registration and 14-check `bun test` harness.
- [2] This document itself: `testing-session-paper-plugin-demo.tex` / `testing-session-paper-plugin-demo.pdf`, generated by the `latex-paper-writer` sub-agent spawned through the plugin’s end-to-end path.